Tipos de datos

Una variable tiene bits pero el tipo dice cómo interpretarlos. 11001100 en uint8_t es 204. Si fuera int8_t es -52.

En Arduino Uno el int normalmente ocupa 2 bytes, en ESP32 ocupa 4. Por eso cuando se necesita el tamaño exacto se usa int8_t, uint8_t, uint16_t, uint32_t.

uint8_t va de 0 a 255. Cuando llega a 255 y sumo 1, vuelve a 0. Los enteros sin signo trabajan dando la vuelta.

Con 200 + 100 el resultado matemático es 300, pero guardado en uint8_t queda 44.

No hay que confiar en el overflow de los tipos con signo. sizeof() sirve para saber cuantos bytes ocupa un tipo.

Cuando hago ~a siendo a de 8 bits, primero puede convertirse a int para la operación. Por eso a veces el resultado impreso tiene muchos bits. Se puede convertir a uint8_t si quiero conservar solo los 8.

Para el bit 31 del ESP32, mejor 1UL << 31, por el asunto del signo del int.

const para valores que no deben cambiar, constexpr se calcula como constante cuando aplica. #define reemplaza texto y enum para nombres de estados.

static en una función hace que la variable conserve valor entre llamadas. volatile se usa cuando la variable puede cambiar desde una interrupción o hardware; no significa que ya sea atómica.

Un error: uint8_t i y condición i < 256, porque nunca va a llegar a 256. Otro error guardar millis() en int.

Tiempo sin delay

Con delay(500), el programa está esperando y no pregunta por el botón durante ese tiempo. Si alguien presiona rápido, no alcanza a verlo.

millis() da tiempo desde que empezó el programa, en milisegundos. Guardarlo en uint32_t o unsigned long según la placa.

ahora = millis(); si (ahora - anterior >= intervalo) hago la tarea y guardo el tiempo.

La resta es importante, porque cuando millis da la vuelta sigue funcionando si se usan tipos sin signo e intervalos adecuados.

Con dos LEDs: uno cambia cada 500 ms y otro cada 130 ms. Cada uno necesita su anterior y su estado, así pueden andar al mismo tiempo sin quedarse esperando.

No es que las dos instrucciones se ejecutan exactamente en paralelo, el loop pasa rápido revisando cada condición.

Pulsadores

Un botón sin resistencia puede dejar la entrada flotando. INPUT_PULLUP mantiene el pin en alto cuando nadie toca el botón.

Botón entre GPIO y GND: sin presionar HIGH, presionado LOW. Parece al revés pero así funciona con pull-up.

GPIO34-39 del ESP32 clásico no llevan pull-up interna. Si uso esos pines debo colocar la resistencia externamente.

El rebote es cuando al presionar, el contacto hace varios 0 y 1 en milisegundos, aunque la persona lo tocó solo una vez.

Antirrebote: aceptar el cambio cuando el estado se mantiene estable 20 o 30 ms. Evitar un delay grande porque otra vez se pierde lo que pasa.

Nivel: sigue presionado. Flanco: acaba de cambiar de HIGH a LOW. Para contar pulsaciones debo detectar el flanco y no sumar cada vuelta que siga en LOW.

Guardar lectura anterior, estado estable y el tiempo del último cambio. Si vuelve a cambiar rápido, esperar.

En el juego de luces se separa leer botón, actualizar posición/velocidad y mostrar LED. El botón puede cambiar velocidad mientras siguen las luces.

Cuando se lleva mucho tiempo presionado, no debería contar muchas veces

La idea de todo esto es que loop siga girando y que ninguna tarea lo deje esperando demasiado.